

UNIDADE II

Programação Orientada a Aspectos (POA) com Spring Boot

Carga Horária: 12 horas-aula

Foco: material conceitual para apresentação em sala

Objetivos da Unidade

Ao final desta unidade, o aluno deverá ser capaz de:

- Identificar interesses transversais em aplicações OO.
- Compreender espalhamento e entrelaçamento de código.
- Utilizar o Spring AOP para implementar aspectos.
- Aplicar aspectos para auditoria, logging, segurança e monitoramento.
- Avaliar quando utilizar POA em aplicações Spring Boot.

1. Introdução

A POO organiza bem o domínio, mas...

Algumas funcionalidades se repetem em várias classes:

- Registro de logs
- Auditoria
- Segurança
- Monitoramento
- Controle transacional
- Tratamento de exceções

Essas são chamadas de **interesses transversais**.

Exemplo sem POA

```
@Service
public class ClienteService {

    public Cliente buscar(Long id) {

        System.out.println("Iniciando busca");

        Cliente cliente = repository.findById(id).orElseThrow();

        System.out.println("Busca finalizada");

        return cliente;
    }
}
```

Lógica de log **misturada** com a lógica de negócio.

2. Interesses Transversais

Interesse Transversal	Aplicação
Logging	Registro de operações
Segurança	Controle de acesso
Auditoria	Rastreabilidade
Monitoramento	Tempo de resposta
Transações	Commit e Rollback
Tratamento de Erros	Captura de exceções

3. Problemas: Scattering e Tangling

Espalhamento (Scattering)

O mesmo código aparece em **diversos locais**.

```
logger.info("Iniciando operação");
```

Essa instrução pode aparecer em dezenas ou centenas de métodos.

Entrelaçamento (Tangling)

```
public Pedido criarPedido() {  
    logger.info("Criando pedido");    // logging  
    validarPermissao();                // segurança  
    Pedido pedido = processarPedido(); // negócio  
    logger.info("Pedido criado");      // logging  
    return pedido;  
}
```

Regra de negócio, segurança e logging **misturados**.

Explicando de forma direta

- **Scattering (espalhamento):** a mesma preocupação técnica fica distribuída em vários pontos do sistema.

Exemplo: chamadas de log repetidas em diversos métodos de vários services.

- **Tangling (entrelaçamento):** várias responsabilidades diferentes ficam misturadas dentro do mesmo método.

Exemplo: no mesmo método há regra de negócio, validação de permissão, logging e tratamento de erro.

Resumo prático:

- **Scattering** → uma preocupação em **muitos lugares**.
- **Tangling** → **muitas preocupações** no mesmo lugar.

4. O que é POA?

A POA é um paradigma que separa os interesses transversais da lógica principal.

Sem POA:

```
PedidoService
├── Regra de negócio
├── Logging
├── Segurança
└── Auditoria
```

Com POA:

PedidoService	→ Regra de negócio
LogAspect	→ Logging
SegurancaAspect	→ Segurança
AuditoriaAspect	→ Auditoria

5. Conceitos Fundamentais

Aspect

Classe que encapsula um interesse transversal.

```
@Aspect  
@Component  
public class LogAspect { }
```

Join Point

Ponto de execução onde o aspecto pode atuar.

(no Spring AOP: execução de método de bean Spring)

Pointcut

Expressão que seleciona quais Join Points serão interceptados.

```
execution(* br.com.exemplo.service.*.*(..))
```

→ Qualquer método, de qualquer classe, no pacote `service` .

Advice

Ação executada quando um Pointcut é atingido.

Tipo	Quando executa
<code>@Before</code>	Antes do método
<code>@After</code>	Após o método (sucesso ou erro)
<code>@AfterReturning</code>	Apenas quando termina com sucesso
<code>@AfterThrowing</code>	Apenas quando ocorre exceção
<code>@Around</code>	Envolve a execução (antes e depois)

Como obter atributos do método

No Advice, use `JoinPoint` ou `ProceedingJoinPoint` para acessar metadados.

```
@Around("execution(* br.unifor.produtosapi.service..*(..))")
public Object inspecionar(ProceedingJoinPoint pjp) throws Throwable {
    MethodSignature sig = (MethodSignature) pjp.getSignature();
    Method metodo = sig.getMethod();

    String nome = metodo.getName();
    Class<?> retorno = metodo.getReturnType();
    Class<?>[] parametros = metodo.getParameterTypes();
    Object[] args = pjp.getArgs();

    return pjp.proceed();
}
```

Pointcut em detalhe (leitura)

```
execution(* br.unifor.produtosapi.service..*(..))
```

- `*` retorno: qualquer tipo
- `service..` pacote: inclui subpacotes
- `*` método: qualquer nome
- `(..)` parâmetros: qualquer quantidade/tipo

Designators úteis:

- `execution`, `within`, `args`, `@annotation`, `this`, `target`

Composição:

- `&&` (E), `||` (OU), `!` (NÃO)

Referências rápidas (Pointcut):

Advice em detalhe (quando usar)

- `@Before` : antes da execução (ex.: validação, log de entrada)
- `@After` : no fim, com sucesso ou erro (ex.: limpeza)
- `@AfterReturning` : somente sucesso, com acesso ao retorno
- `@AfterThrowing` : somente erro, com acesso à exceção
- `@Around` : envolve tudo e usa `proceed()`

Exemplo de binding:

```
@AfterReturning(pointcut = "execution(* ..service..*(..))", returning = "resultado")
public void aposSucesso(Object resultado) { }

@AfterThrowing(pointcut = "execution(* ..service..*(..))", throwing = "ex")
public void aposErro(Exception ex) { }
```

Referências rápidas (Advice):

6. Spring AOP

Dependência Maven

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-aspectj</artifactId>  
</dependency>
```

Como o Spring aplica o aspecto

- Cria **proxies** para beans gerenciados (`@Service` , `@Component` , `@Repository` , `@RestController`).
- Ao chamar o bean pelo container, o proxy executa o advice **antes/depois** do método alvo.

7. Tipos de Advice — Before e After

```
@Before("execution(* br.com.exemplo.service.*.*(..))")  
public void logAntes() {  
    System.out.println("Método iniciado");  
}
```

```
@After("execution(* br.com.exemplo.service.*.*(..))")  
public void logDepois() {  
    System.out.println("Método finalizado");  
}
```

AfterReturning e AfterThrowing

```
@AfterReturning(  
    pointcut = "execution(* br.com.exemplo.service.*.*(..))",  
    returning = "resultado")  
public void sucesso(Object resultado) {  
    System.out.println(resultado);  
}
```

```
@AfterThrowing(  
    pointcut = "execution(* br.com.exemplo.service.*.*(..))",  
    throwing = "ex")  
public void erro(Exception ex) {  
    System.out.println(ex.getMessage());  
}
```

Around — o mais poderoso

```
@Around("execution(* br.com.exemplo.service.*.*(..))")
public Object monitorar(ProceedingJoinPoint joinPoint)
    throws Throwable {

    long inicio = System.currentTimeMillis();

    Object retorno = joinPoint.proceed();

    long fim = System.currentTimeMillis();

    System.out.println("Tempo: " + (fim - inicio));

    return retorno;
}
```

8. Exemplo de Logging com Spring AOP

```
@Service
public class ProdutoService {
    public void cadastrar() {
        System.out.println("Produto cadastrado");
    }
}
```

```
@Aspect
@Component
public class LogAspect {
    @Before("execution(* br.com.exemplo.service.*(..))")
    public void registrarLog() {
        System.out.println("Executando método...");
    }
}
```

Resultado:

9. Aspectos Baseados em Anotações

```
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface Auditavel { }
```

```
@Service
public class ClienteService {
    @Auditavel
    public void excluir(Long id) { ... }
}
```

```
@Aspect
@Component
public class AuditoriaAspect {
    @Before("@annotation(Auditavel)")
    public void auditar() {
        System.out.println("Operação auditada");
    }
}
```

10. Estudo de Caso — Sistema de Pedidos

Requisitos:

- Registrar logs
- Auditar operações
- Medir tempo de execução

```
@Service
public class PedidoService {
    public void gerarPedido() {
        System.out.println("Pedido gerado");
    }
}
```

O PedidoService **não contém** código de log, auditoria ou monitoramento.

Aspectos para PedidoService

```
@Aspect @Component
public class LogAspect {
    @Before("execution(* *..PedidoService.*(..))")
    public void log() { System.out.println("Início"); }
}
```

```
@Aspect @Component
public class MonitoramentoAspect {
    @Around("execution(* *..PedidoService.*(..))")
    public Object medirTempo(ProceedingJoinPoint pjp) throws Throwable {
        long inicio = System.currentTimeMillis();
        Object retorno = pjp.proceed();
        long fim = System.currentTimeMillis();
        System.out.println("Tempo: " + (fim - inicio));
        return retorno;
    }
}
```

11. Boas Práticas

Utilize Aspectos Para:

- Logging
- Auditoria
- Segurança
- Monitoramento / Métricas
- Transações

Evite Aspectos Para:

- Regras de negócio
- Cálculos de domínio
- Validações de domínio
- Processamento principal da aplicação

Desvantagens do uso de aspectos

- Comportamento menos explícito ao ler só o service
- Debug mais trabalhoso com proxy/interceptação
- Pointcut amplo pode interceptar métodos indevidos
- Overhead adicional por chamada (proxy + advice)
- Conflitos de ordem entre múltiplos aspectos

Mitigação:

- Pointcuts específicos e bem nomeados
- Advice leve (evitar lógica pesada)
- Ordem dos aspectos documentada
- Métricas para monitorar impacto real

12. Limitações do Spring AOP

Trabalha apenas com **Beans** gerenciados pelo Spring.

Não intercepta:

- Objetos criados com `new` fora do container
- Métodos privados
- Construtores
- Campos/Atributos

Self-invocation

```
@Service
public class PedidoService {
    public void processar() {
        validar(); // NÃO passa pelo proxy – advice não dispara
    }
    public void validar() { ... }
}
```

Chamada interna **não passa pelo proxy** — o advice pode não disparar.

Para interceptar construtores ou self-invocations: usa-se **AspectJ completo**.

13. Exemplo no Projeto da Turma

Arquivo: backend/produtos-api/src/.../aspect/ObservabilidadeAspect.java

```
@Aspect
@Component
public class ObservabilidadeAspect {

    @Pointcut("within(br.unifor.produtosapi.controller..*) " +
              "|| within(br.unifor.produtosapi.service..*)")
    public void camadasApi() {}

    @Around("camadasApi()")
    public Object registrarTempoEStatus(ProceedingJoinPoint joinPoint)
        throws Throwable {
        // log INICIO, executa método, log FIM com tempo e status
    }
}
```

Demo em Sala

```
cd backend/produtos-api  
./mvnw spring-boot:run
```

```
curl http://localhost:8080/tipos
```

No console:

```
INICIO metodo=listar args=[]  
FIM metodo=listar tempoMs=12 sucesso=true
```

Forçar erro → `sucesso=false` com tipo e mensagem da exceção.

O service/controller não tem código de log — separação real.

14. Roteiro de Apresentação (35–45 min)

1. Problema real: código repetido (5 min)
2. Conceitos: Aspect, Join Point, Pointcut, Advice (10 min)
3. Spring AOP e proxies (8 min)
4. Limitações e armadilhas, incluindo self-invocation (7 min)
5. Demo no projeto da turma (10–15 min)

15. Exercício Prático

Desenvolva uma API Spring Boot com:

Serviços: `UsuarioService` , `ProdutoService` , `PedidoService`

Aspectos: `LogAspect` , `AuditoriaAspect` , `TempoExecucaoAspect`

Requisitos:

1. Registrar entrada em todos os métodos.
2. Registrar saída dos métodos.
3. Medir tempo de execução.
4. Auditar métodos anotados com `@Auditavel` .
5. Não incluir código de log dentro dos serviços.

16. Questões para Fixação

1. O que são interesses transversais? O que os caracteriza?
2. Explique com exemplos a diferença entre scattering e tangling.
3. O que é um Aspect? Um Join Point? Um Pointcut?
4. O que é um Advice? Quais são os tipos existentes?
5. Qual a diferença entre `@Before` e `@Around` ?
6. Por que `@Around` é tão usado em observabilidade?
7. O que é self-invocation e por que pode impedir a interceptação?
8. Quando utilizar Spring AOP? Quando evitar?
9. Quais são as limitações do Spring AOP?

Resumo da Unidade

- Interesses transversais e os problemas de scattering e tangling.
- Conceitos fundamentais: Aspect, Join Point, Pointcut, Advice.
- Spring AOP e o modelo baseado em proxies.
- Tipos de Advice: Before, After, AfterReturning, AfterThrowing, Around.
- Aspectos baseados em anotações (`@Auditavel`).
- Boas práticas e limitações (incluindo self-invocation).
- Exemplo prático com `ObservabilidadeAspect` no projeto da turma.

A POA **complementa** a arquitetura separando preocupações transversais — não substitui modelagem de domínio.